

Lab 1: Timing Honestly, and the Memory Wall

Measure a program correctly, then watch loop order change its speed by a factor of eight without changing a single arithmetic operation.

Prepared by **Dr. Abhijith Anandakrishnan**

Assistant Professor, Amrita School of AI

DURATION

One 2-hour block

PREREQUISITE

Unit 0

SUBMIT

lab01-<ROLLNO>.pdf

MARKS

10

OFFERING

Odd Semester 2026-27

The point of this lab

Everything else in this course depends on being able to answer “is this faster?” honestly. That turns out to be harder than it looks, and most people get it wrong in one of four specific ways.

Then, once you can measure, you will see the memory wall with your own eyes.

BY THE END OF THIS LAB YOU WILL BE ABLE TO

- Choose the right clock, and explain why `clock()` is the wrong one.
- Design a measurement with warm-up, repetition and a stated statistic.
- Demonstrate and explain an eight-fold difference caused only by loop order.
- Find the size at which your machine's cache stops helping.
- Report a performance result that somebody else could reproduce.

Part 1: The four ways to measure wrongly

1.1 The wrong clock

Compile and run this:

```
#include <omp.h>
#include <stdio.h>
#include <time.h>

int main(void)
{
    const long n = 200000000;
    double sum = 0.0;

    clock_t c0 = clock();
    double w0 = omp_get_wtime();

#pragma omp parallel for reduction(+ : sum)
    for (long i = 0; i < n; i++) sum += 1.0 / (double)(i + 1);

    double cpu = (double)(clock() - c0) / CLOCKS_PER_SEC;
    double wall = omp_get_wtime() - w0;

    printf("sum=%.10f  cpu=%.3fs  wall=%.3fs  ratio=%.2f\n",
        sum, cpu, wall, cpu / wall);
    return 0;
}
```

```
gcc -O2 -fopenmp lab01a.c -o lab01a -lm
for t in 1 2 4 8; do OMP_NUM_THREADS=$t ./lab01a; done
```

Exercise 1.1. Record `cpu`, `wall` and `ratio` at each thread count. What does the ratio converge to, and why? If a classmate reported the `cpu` figure as their runtime, what speedup would they claim at 8 threads, and what is the truth?

1.2 The optimiser deleted your benchmark

```
t0 = now();
for (i = 0; i < n; i++) { double x = expensive(a[i]); } /* x unused */
t1 = now();
```

Exercise 1.2. Write a loop whose result is never used, time it at `-O0` and at `-O2`, and report both. Then make the result *escape* (accumulate it and print it) and time again. Explain the three numbers.

1.3 The first run

Exercise 1.3. Time the same computation ten times in a row within one process. Plot the ten timings. How much slower is run 1 than run 10? Name two reasons.

1.4 One measurement

Exercise 1.4. Take your ten timings. Report the minimum, the mean, the maximum and the standard deviation. Which single number should you report, and why is it not the mean?

DEFINITION · THE RULE FOR THIS COURSE

Warm up once, run at least five times, report the **minimum**, and state the spread. Interference from the rest of the system can only slow a run down, so the minimum is the closest you got to measuring your program rather than the machine's other occupants.

Part 2: Loop order

Now the memory wall.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

static double now(void)
{
    struct timespec t;
    clock_gettime(CLOCK_MONOTONIC, &t);
    return t.tv_sec + 1e-9 * t.tv_nsec;
}

int main(int argc, char **argv)
{
    int n = (argc > 1) ? atoi(argv[1]) : 4096;
    double *a = malloc((size_t)n * n * sizeof *a);
    for (long i = 0; i < (long)n * n; i++) a[i] = 1.0;

    double s1 = 0.0, s2 = 0.0, t0, t_row, t_col;

    t0 = now();
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++) s1 += a[(long)i * n + j];    /* row order */
    t_row = now() - t0;

    t0 = now();
    for (int j = 0; j < n; j++)
        for (int i = 0; i < n; i++) s2 += a[(long)i * n + j];    /* column order */
    t_col = now() - t0;

    printf("n=%d row=%.3fs col=%.3fs ratio=%.2f (%.1f %.1f)\n",
        n, t_row, t_col, t_col / t_row, s1, s2);
    free(a);
    return 0;
}
```

Exercise 2.1. Run at $n = 512, 1024, 2048, 4096$. Tabulate the ratio. Both loops perform exactly n^2 additions and read exactly the same values. Explain the ratio in terms of cache lines, and say why it grows with n .

Exercise 2.2. A `double` is 8 bytes and a cache line is 64 bytes. Compute, for each loop, the number of bytes of memory traffic per useful element. Does the ratio you predict match the ratio you measured? If not, suggest why the hardware prefetcher might make the gap larger than the simple calculation.

Exercise 2.3. Confirm it with counters:

```
perf stat -e cache-references,cache-misses,LLC-load-misses ./lab01b 2048
```

Report the miss counts. (If `perf` is restricted on your machine, run this part on the cluster inside an `srun`.)

Part 3: Finding your cache sizes

Read a fixed number of elements from arrays of increasing size, and watch the time per element jump each time you fall out of a cache level.

Exercise 3.1. Write a program that, for working-set sizes from 4 KB to 256 MB (doubling each time), performs the same total number of reads and reports nanoseconds per access. Plot ns-per-access against working-set size on a log x axis.

You should see three or four plateaus with steps between them. **Mark on your plot where you think L1, L2, L3 and DRAM begin.** Then check against the truth:

```
lscpu | grep -i cache
```

Exercise 3.2. Do your measured step positions match the reported cache sizes? If a step appears earlier than the cache size suggests, give a reason.

PITFALL · LET THE COMPILER HELP YOU, NOT FOOL YOU

Use a dependent access pattern, for example a pointer-chase or an index that depends on the previous value, so the processor cannot simply prefetch the whole array and hide the latency you are trying to measure. A simple `sum += a[i]` measures **bandwidth**; a chase measures **latency**. Say which you measured.

Part 4: One honest report

Exercise 4.1. Take the best version of your Part 2 program and write a performance report of one paragraph that another student could reproduce exactly. It must state:

- machine (model, cores, clock, cache sizes)
- compiler and version, and the exact flags
- problem size
- number of warm-up and timed repetitions
- the statistic reported
- the result, with its spread

This paragraph is the template for every performance claim you make for the rest of the course, including in your mini project.

What to submit

A single PDF, `lab01-<ROLLNO>.pdf` :

1. Answers to all exercises, with the numbers you actually measured.
2. Your source files as text.
3. Two plots: the ten-run timing spread (Part 1) and the cache-size staircase (Part 3), both with labelled axes.
4. The reproducible report paragraph from Part 4.

MARKS	FOR
3	Part 1: all four measurement errors demonstrated and explained
3	Part 2: correct ratio measured, and explained in terms of cache lines
2	Part 3: a staircase plot with the levels identified
1	Part 4: a genuinely reproducible report paragraph
1	Presentation: text not screenshots, plots labelled, numbers with units